1.1 Project Charter & Vision

Project Name: *LibTracker*

Team: *Rebs Dev*

Members: *Sirac Ketenoğlu, Emre Turan, Barış Küçükkaya, Reis Yıldız*

Date: 22/03/2026

Purpose

The primary purpose of LibTracker is to monitor and manage and display the live occupancy levels of university libraries and study spaces.

Scope

- Displaying the real time occupancy and busyness status of library areas.
- Allowing students to submit live feedback regarding current capacity and correctness of statistics.
- Generating and displaying hourly and daily usage statistics for future planning.

Problem & Motivation

- **What problem are you solving?**

  We are solving the problem of unpredictable busyness and wasting time to find available spots in university libraries.

- **Who experiences it, and why is it painful now?**

  University students experience this problem. It is really demotivating due to the large student population competing for a limited amount of study space. This is resulting in significant time and energy wasted searching for an empty seat.

- **What is the context (where/when does it happen)?**

  This problem happens in university libraries and study areas during active working hours, especially during exam periods.

- **What is the intended impact if solved?**

  The intended impact is to save students' valuable time by allowing them to check real time availability remotely, enabling them to make better decisions before commuting to the library.

Target Users and Stakeholders

- **Primary users**
  - **University Students** - To check live library busyness levels and plan their study schedules efficiently without wasting time.
  - **Students at Exam Periods** - To quickly locate quieter, less crowded study areas when stress levels and population are really high.
- **Other stakeholders**
  - **Library Management** - To analyze library usage patterns and allocate library resources, staff, and spaces more effectively.
  - **University Administration** - To use occupancy statistics to improve overall student productivity and make decisions according to real data for opening new study areas.

Resources

- **Resource 1:** Development environment for full stack implementation (*Python* for backend; *JavaScript*, *HTML*, and *CSS* for frontend).
- **Resource 2:** *MySQL Database* system for data storage and management.
- **Resource 3:** *GitHub* repository for teamwork and version control.

Timeline

- **Week 1-2:** Project Planning and Requirement Analysis
- **Week 3-5:** Understanding Personas, User stories and Features; Making architectural decisions; Creating necessary diagrams
- **Week 6-9:** Creating collobrative enviroment for development and establishing foundations of Core Development and Deployment
- **Week 10-12:** System Testing, Debugging, and Quality Controls
- **Week 13-14:** Final Documentation, Preparing Report and Project Presentation

Potential Risks, Dependencies and Constraints

- **Data Reliability:** Gathering live and accurate data is challenging, because it relies heavily on user feedbacks.
- **Data Integrity:** Incorrect feedbacks could damage the system's accuracy and decrease user trust.
- **Market Competition:** Existing scheduling or tracking applications might shadow our product.
- **Budget Constraint:** Market competiton requires budget for advertisement.

Product Vision

For university students **who** struggle to find available study spaces, **the** LibTracker **is an** occupancy tracking application **that** displays the real time busyness of libraries. **Unlike** try and error methods or competitor apps, **our product** saves students' time by providing reliable data and making study spaces highly accessible.

1.2 Personas, Scenarios, User Stories & Feature List

User Personas

Damla, a senior law student who lives far from school and on exam period

Damla, age 22, is a senior law student living in Izmir. She is the daughter of a retired banker and an administrative worker, and she is known for being very disciplined. Because her home is far from the campus and it is exam period, time management is important for her, and she plans her day minute by minute. Therefore, if she goes there and cannot find a seat, she wastes 2 hours on the road. Damla is very disciplined so efficiency and predictability is very important for her. She passionately believes that time should not be wasted. She is particularly interested in using the LibTracker system to check real time occupancy before leaving home to make a smart decision and use her time well.

Barış, a computer engineering student who cannot focus on loud places and values data integrity

Barış, age 23, is a Computer Engineering student living in Istanbul. As the son of an engineer and a librarian, he cares a lot about accurate data and sharing information with others. He spends most of his day doing deep research in libraries and is very careful in his academic life. During study sessions, crowds and noise are a big problem for his focus. So, he needs to see the population in the library. Barış's engineering background means that he is confident in digital systems and data integrity. He believes that collaborative data sharing can greatly enhance the campus experience for everyone. He is particularly interested in using the LibTracker system not just to find a empty seat, but to help the system stay accurate by sharing current occupancy to help other students get the right information.

Can, a business student who likes studying in a group

Can, age 20, is a business student living near the campus. He is highly social, tends to procrastinate, and usually studies at the last minute with a group of 3-4 friends. He comes from a family of merchants and values teamwork and quick problem-solving.

Can’s practical mindset means he strongly dislikes inefficient tasks, like walking aimlessly between different faculties and reading rooms late at night just to find a large table with a big group. He believes that technology should eliminate unnecessary physical effort. He is particularly interested in using the LibTracker system to quickly see which specific study rooms have enough empty seats for his whole study group before walking across the huge campus.

Zeynep, a head librarian who wants to manage resources better

Zeynep, age 45, is a head librarian at the university's main campus library. She has worked in library administration for over 12 years and is deeply familiar with the daily struggles of managing a large facility. She constantly deals with students complaining about the lack of seats and struggles to know when to schedule cleaning staff without disturbing the study environment.

Zeynep’s administrative experience means she values structured data and organized workflows. She passionately believes that a well managed, quiet library environment is crucial for academic success. She is particularly interested in using the LibTracker system's backend statistics to view daily peak hours and overall occupancy trends, allowing her to manage resources and staff more effectively.

Usage Scenarios

- **Scenario Damla: The Commuter's Decision**

  Damla is studying for her upcoming law exams. She lives far from campus and knows that if she travels to the library and finds it full, she will waste 2 hours on the road. Before leaving her house, she opens LibTracker to check the real time occupancy. Seeing that the main library is completely full but the law faculty reading room has a lot of empty seats, she heads directly to the faculty room, so her day continues predictable and efficient.

- **Scenario Barış: Focus and Data Contribution**

  Barış needs to do deep research for his computer engineering project but gets easily distracted by noise and crowds. He uses LibTracker to find a reading room with a low population. Once he arrives and settles into the quiet room, he uses the app to submit the current occupancy status. By doing this, he satisfies himself for maintaining data integrity and helps the system to stay accurate for other students.

- **Scenario Can: Last Minute Group Study**

  Can and his 3 friends decide to study late at night for their business exam. Instead of walking for no reason across the huge campus in the cold to find a large table for the group, Can opens LibTracker. He checks the specific study rooms and finds one that has enough empty seats for all four of them. They walk directly to that specific room, saving physical effort and time.

- **Scenario Zeynep: Data-Driven Library Management**

  Zeynep needs to schedule the cleaning staff for the upcoming week but wants to avoid disturbing the students during their study sessions. She accesses the LibTracker backend statistics to check the daily peak hours and overall occupancy trends. She notices that the library is consistently at its emptiest between 8:00 AM and 9:30 AM, so she schedules the cleaning team exactly for that time window, optimizing her resource management.

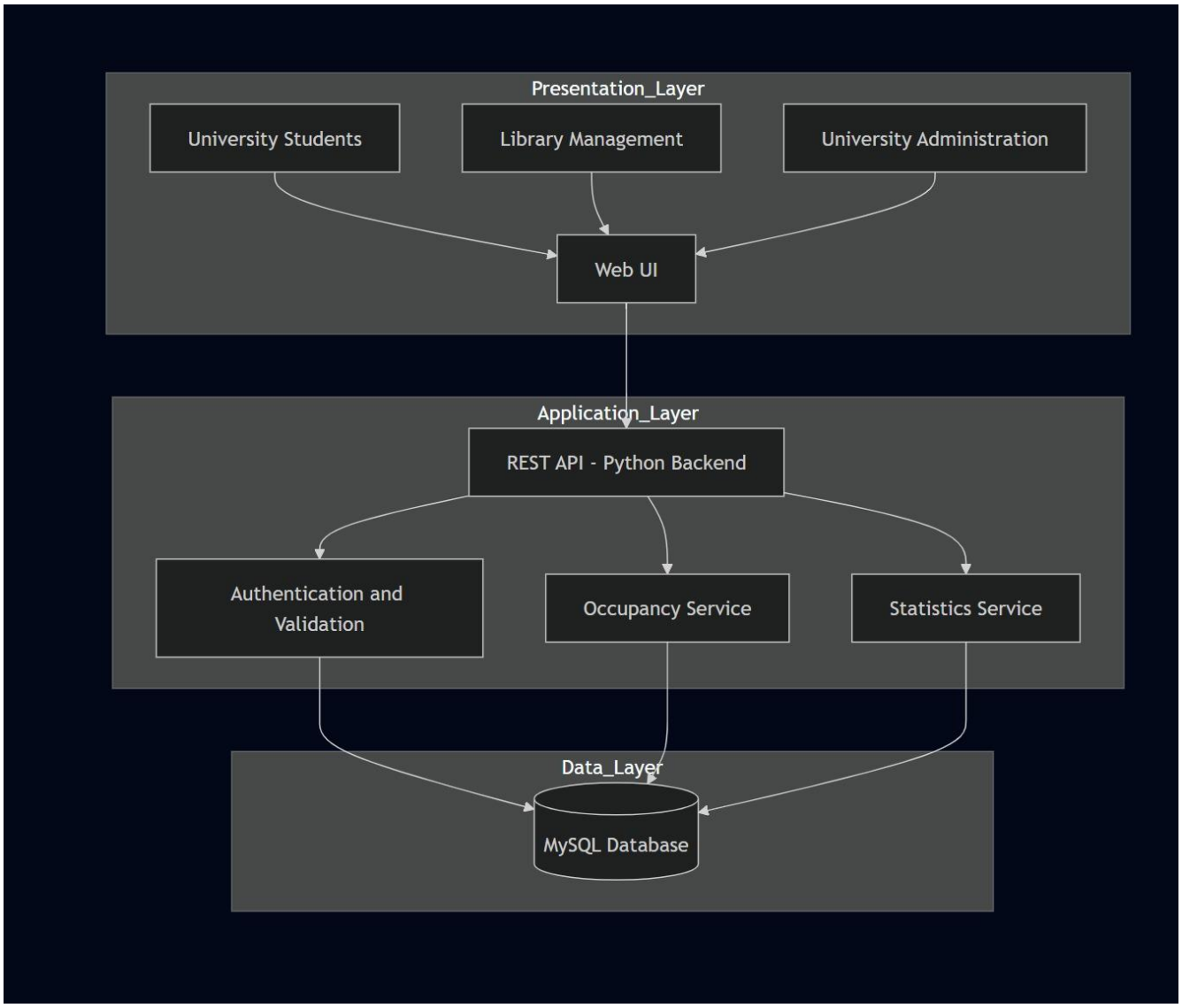
User Stories

- **As a** student living far from campus, **I want to** check real time occupancy before leaving home **so that** I don't waste hours commuting to a full library.
- **As a** student in exam period, **I want to** reserve seat **so that** i can start studying immediatly.
- **As a** student who cannot focus in loud places, **I want to** see the population levels in different areas **so that** I can find a quiet place for deep research.
- **As a** student who values data integrity, **I want to** share the current occupancy status **so that** I can help keep the system accurate for everyone.
- **As a** student studying with a group, **I want to** see specific room capacities **so that** we can quickly find a space large enough for all my friends without walking aimlessly.
- **As a** head librarian, **I want to** view daily peak hours and overall occupancy trends **so that** I can schedule cleaning staff efficiently without disturbing students.
- **As** library management, **I want to** analyze how students use the library to manage resources better.
- **As** university management, **I want to** see occupancy statistics to make plans that help students be more productive.
- **As a** developer, **I want to** build a system that prevents fake feedback **so** the app's data stays correct and reliable.
- **As a** developer, **I want to build** a system that gives access to spesific roles **so** difference can be made for administration.

Prioritised Feature List

1. **Real Time Occupancy Dashboard:** Displays the current busyness and population levels of various libraries and specific study areas.
2. **User Feedback System:** A feature that allows verified students currently in the library to submit and update the active occupancy status to ensure data integrity.
3. **Detailed Area Segmentation:** Segmentation between library data and specific study areas so groups can see exactly where there is space.
4. **Backend Statistics & Analytics Dashboard:** Provides visual charts on daily peak hours and overall occupancy trends for administrative use.
5. **User Authentication & Validation:** Secure login system to ensure that feedback is submitted only by verified university students, maintaining the accuracy and reliability of the data.
6. **Study Area Reservation System:** A module allowing students to book seats or rooms in advance during peak exam periods, ensuring a guaranteed spot.
7. **Role Based Access Control:** A security and interface feature that provides different views and permissions based on the user's role (Student, Library Management, or University Administration), ensuring that each user accesses only the data and tools relevant to them.
8. **Historical Data & Reports:** An analytical tool enabling library and university management to generate, visualize, and download comprehensive reports on long term usage to make better decisions for resource and staff planning.

1.3 Architecture Diagram



Selected Architectural Style: Layered Architecture

The LibTracker system is depending on a Layered Architecture. This architectural style was selected because it provides a strict separation of services, making the system highly understandable, straight forward to implement, and a lot easier to maintain compared to more complex architectures. It perfectly meets the team's development expectations and allows for efficient management of the system's core attributes: Simplicity, Accuracy, and Scalability.

Structural Organisation

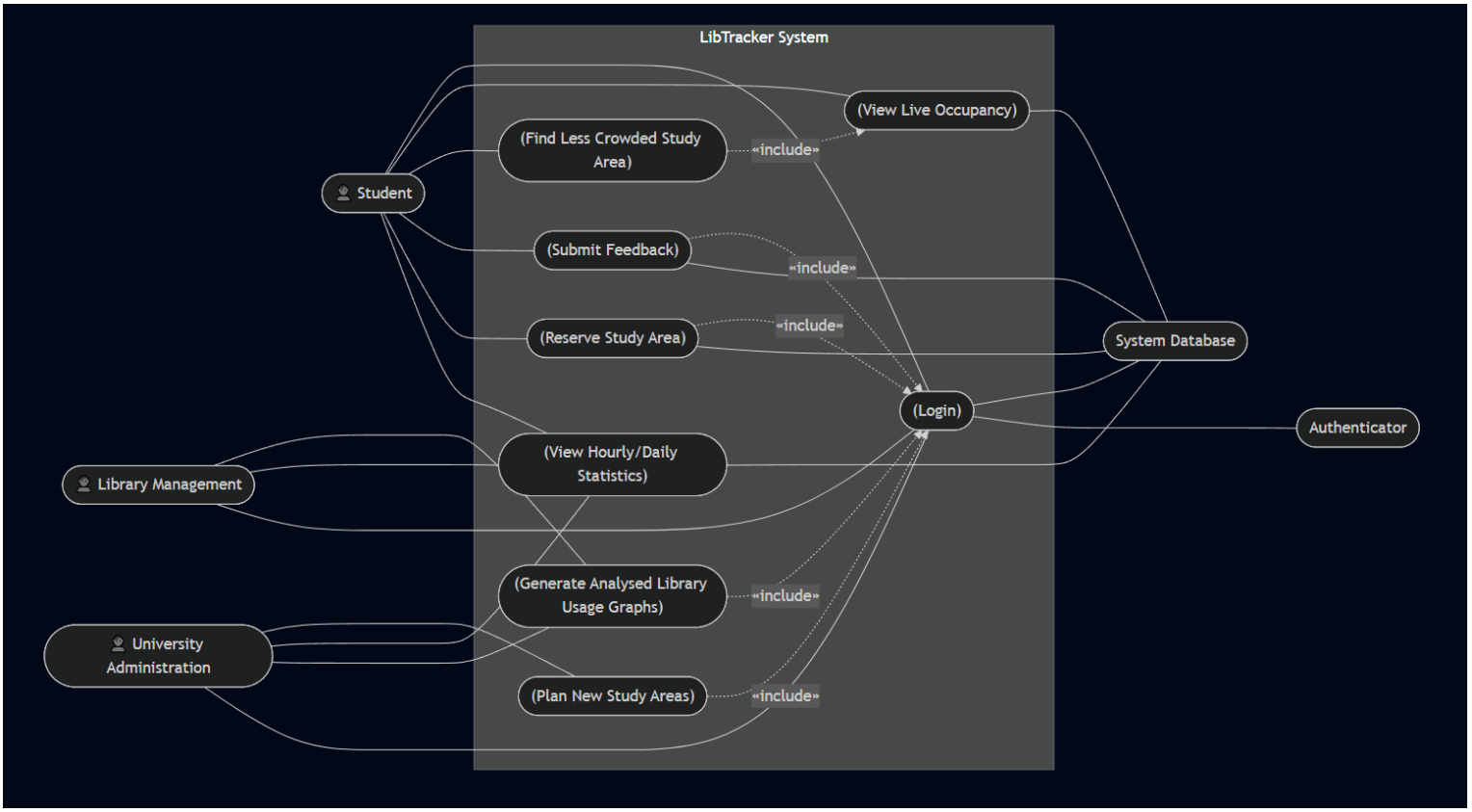
The system is divided into three main layers, each responsible for a different aspect of the application:

- **Presentation Layer:** This layer consists of the Web User Interface. It is designed to be really understandable, three distinct user groups with varying access levels: University Students, Library Management, and University Administration. This separation ensures *Simplicity* by providing understandable UI, user friendly experiences for each role.
- **Application Layer:** This layer is the core of the system, this layer is developed using a Python Backend that exposes a REST API. It handles all rules and data processing through these components:
    - *Authentication and Validation:* Secures the system and verifies user identities to prevent fake feedback, ensuring data *Accuracy*.
    - *Occupancy Service:* Processes real time feedback and seating data.
    - *Statistics Service:* Groups historical data to generate insights.
- **Data Layer:** This layer relies on MySQL Database. It provides persistent, reliable data storage for user credentials, real time occupancy logs, and historical analytics. Database connection ensures the system's *Scalability*, allowing it to handle overloads in traffic without crashing.

1.4 UML Diagrams

The following UML diagrams shows the behavioural and structural design of the LibTracker system, provides all functional requirements and system interactions that are properly mapped before implementation.

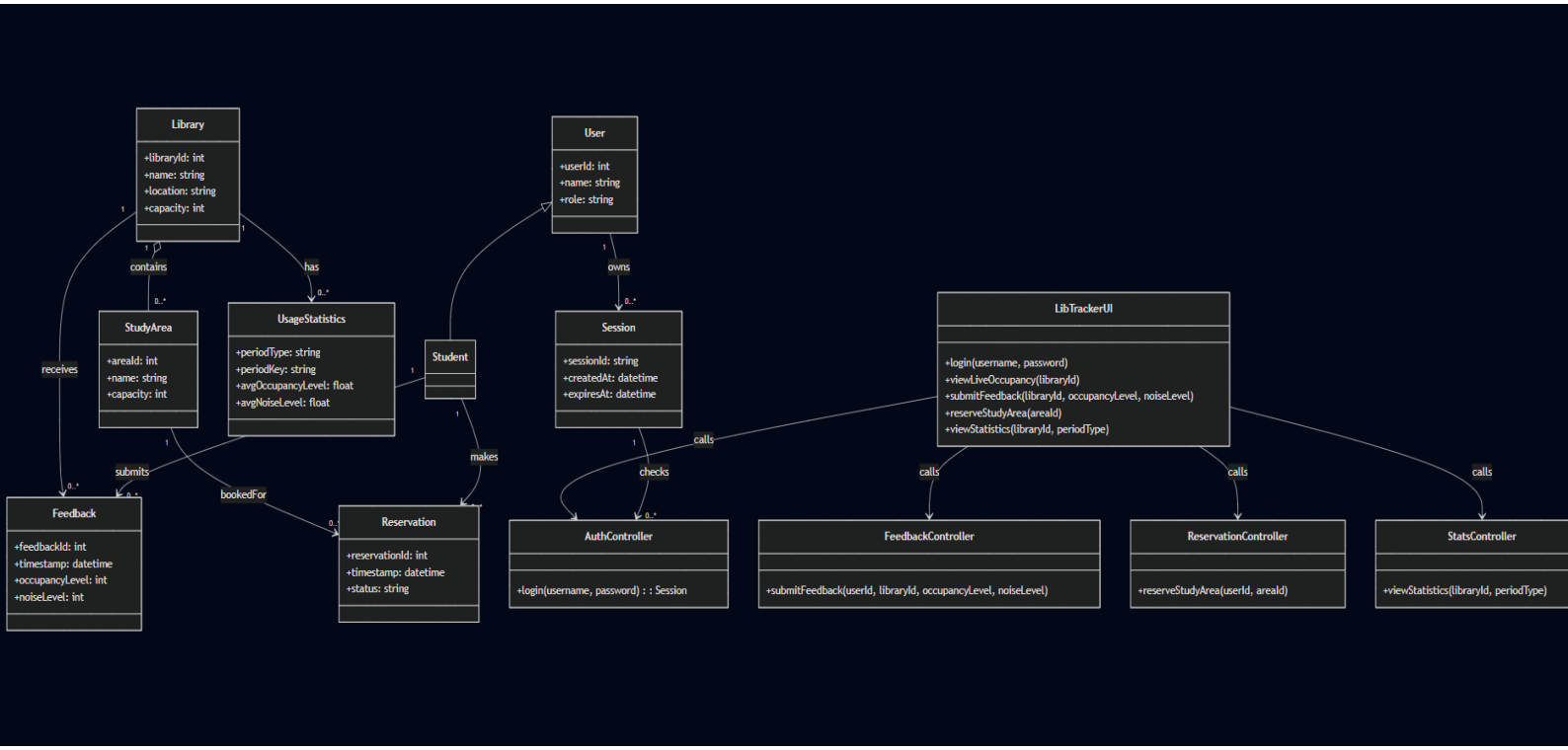
Use Case Diagram



The Use Case diagram shows the main interactions between the system's actors and its functions.

- **Actors:** The system identifies three primary human actors (Student, Library Management, University Administration) and two system actors (Authenticator, System Database).
- **Main Workflows:** Students can interact with the system to "View Live Occupancy", "Find Less Crowded Study Area", "Submit Feedback", and "Reserve Study Area". Library Management and University Administration can interact with use cases such as "Generate Analysed Library Usage Graphs", "View Hourly/Daily Statistics", and "Plan New Study Areas".
- **Dependencies:** Important actions, such as submitting feedback or reserving a study area, include a <<include>> relationship with the "Login" use case, guarantees that only verified users can use system data via the Authenticator.

Class Diagram

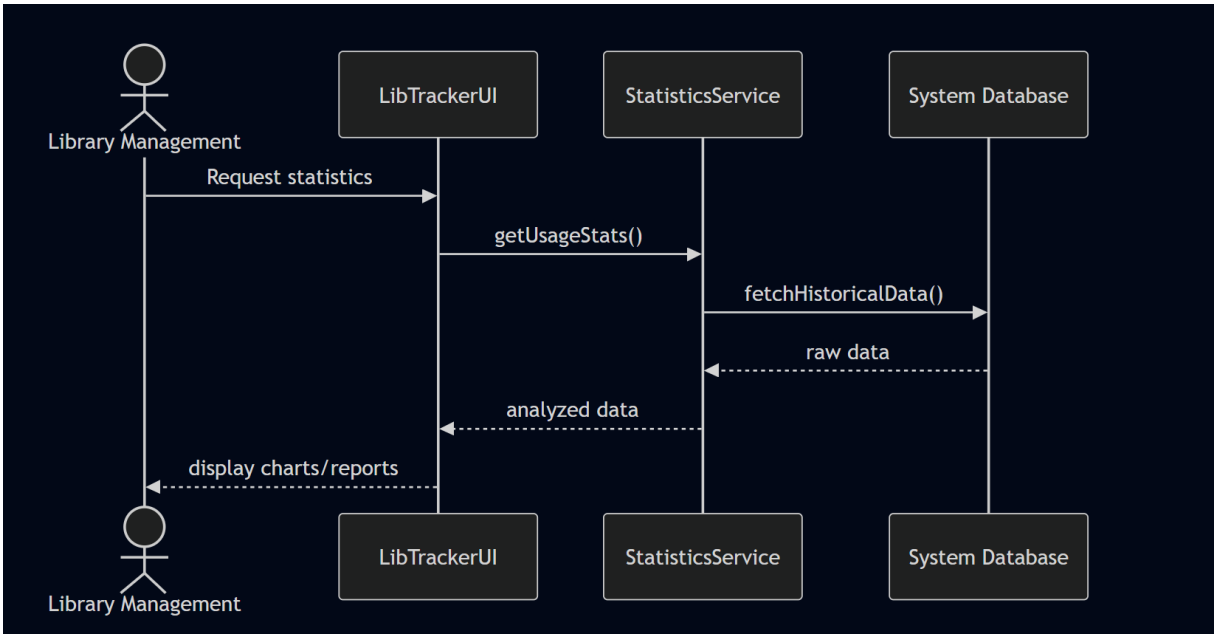


The Class diagram provides a view of the applications structure, gives details of the key entities, their attributes, functions, and relationships.

- **Entities:** The main data models include Library, StudyArea, User (with a specialized Student subclass as a role), Feedback, Reservation, and UsageStatistics.
- **Controllers:** To maintain a clean architecture, the system has some controller classes (AuthController, FeedbackController, ReservationController, and StatsController) managed by a central LibTrackerUI class.
- **Relationships:** The diagram shows simple but clean relations, such as a Library containing multiple StudyAreas, and a Student generating multiple Feedback or Reservation instances. The controllers act as bridges between the UI and database.

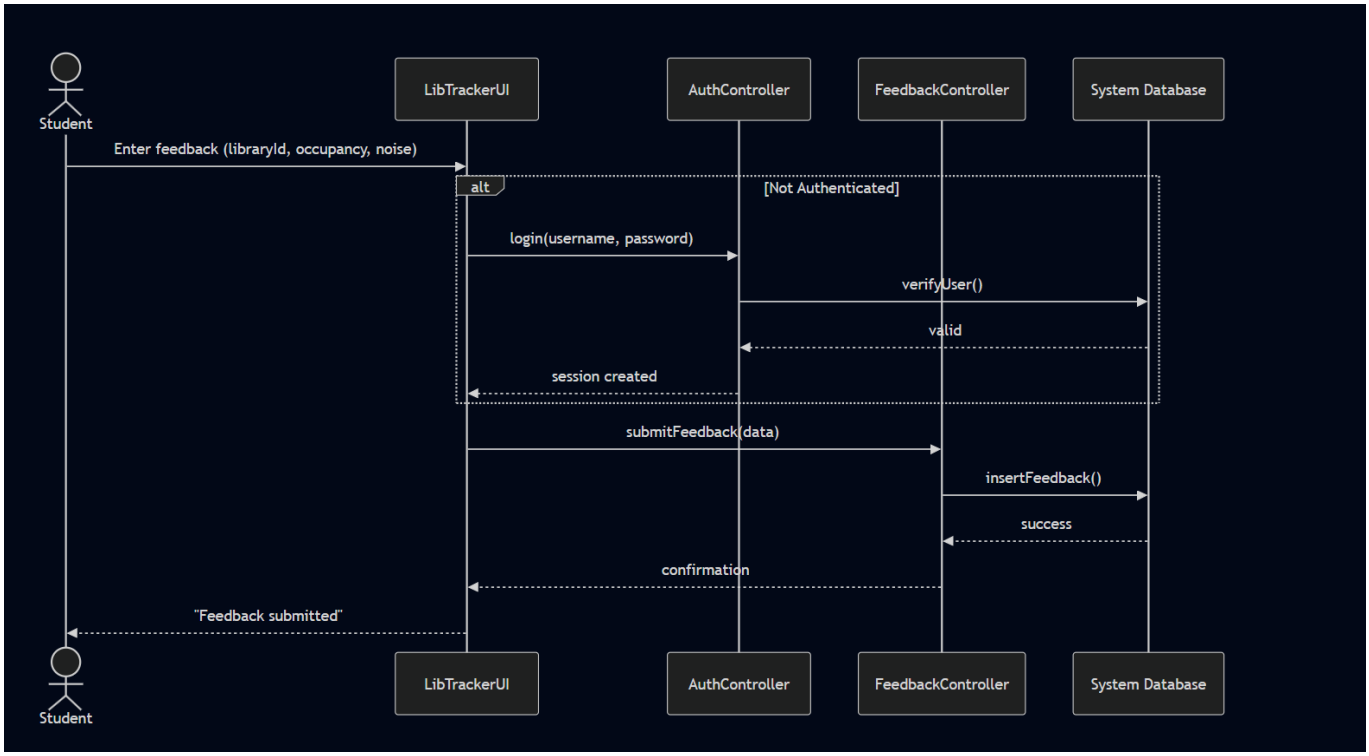
Sequence Diagrams

1.View Hourly/Daily Statistics Sequence Diagram



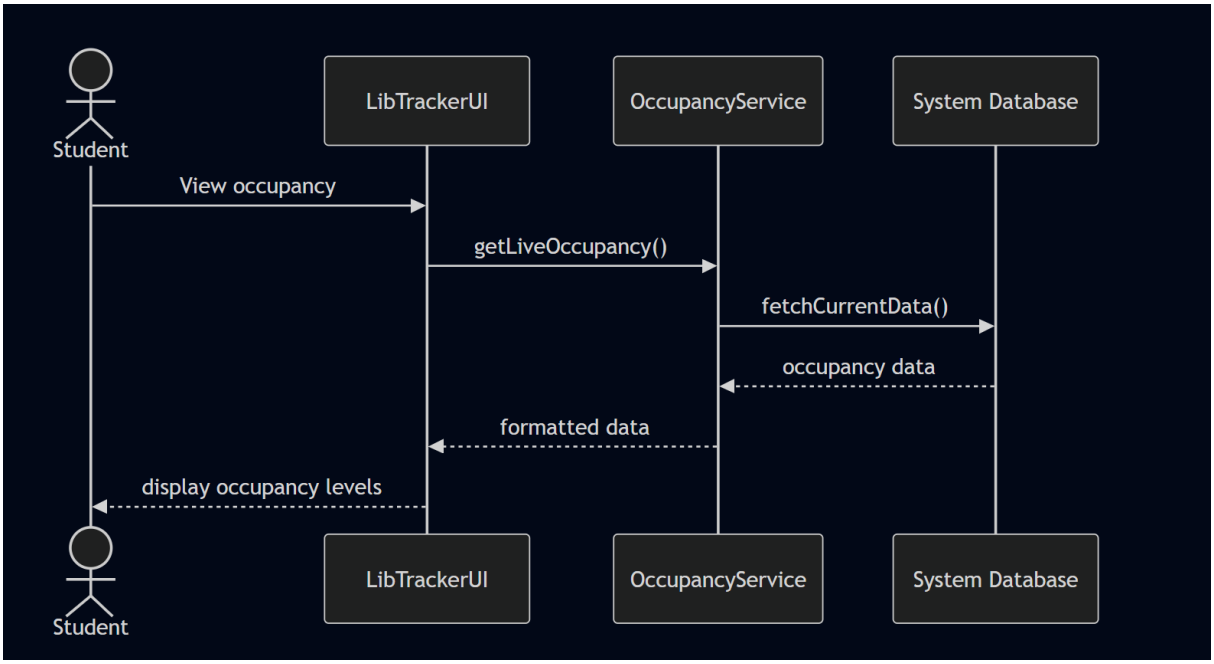
This sequence diagram shows the workflow for the "Library Management" actor retrieving historical analyse data. The user requests statistics through the LibTrackerUI, which then calls the StatisticsService. The service fetches the historical raw data from the System Database, processes it into analyzed data, and returns it to the UI to be displayed as charts and reports.

2. Submit Feedback Sequence Diagram



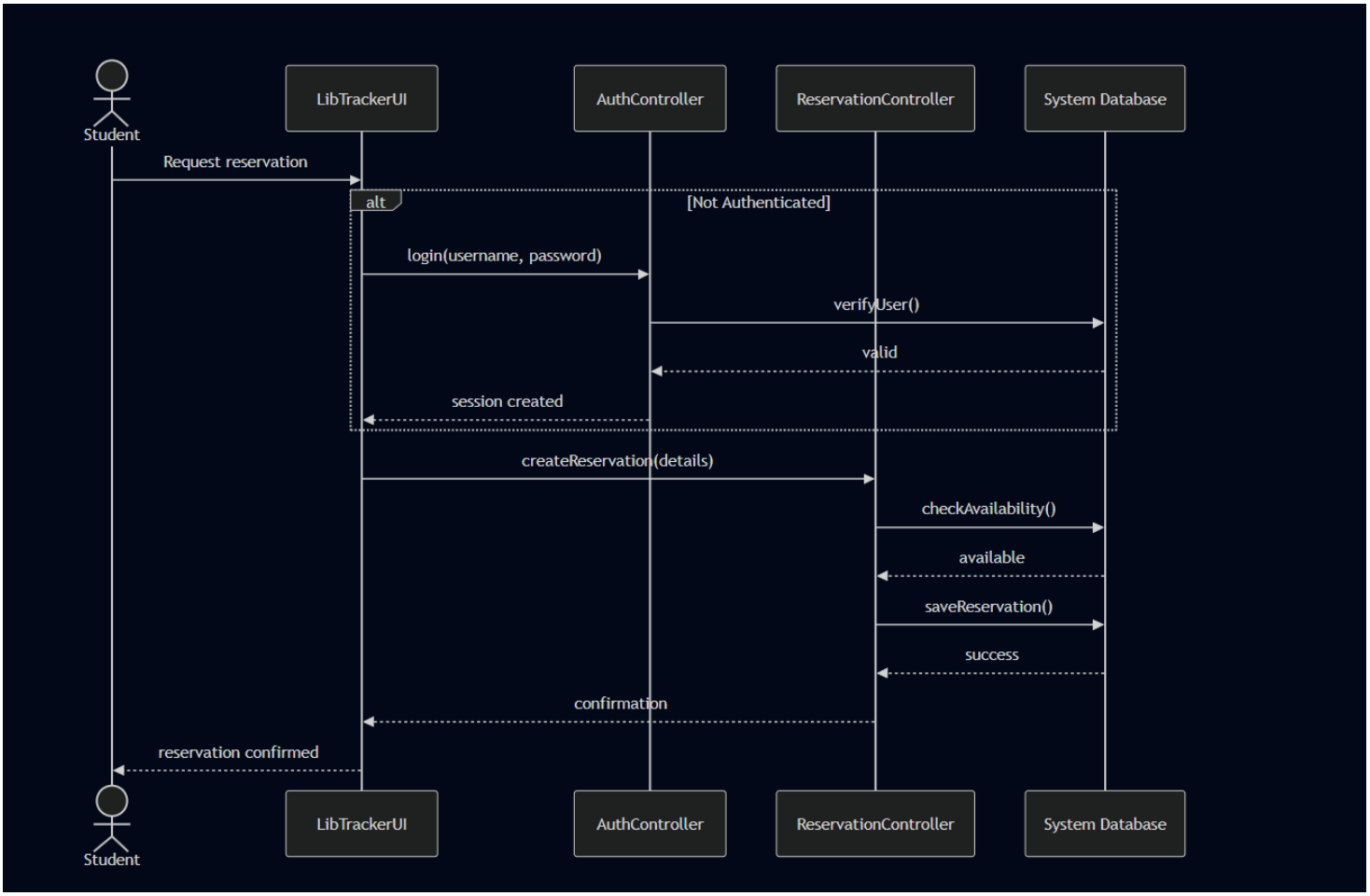
This sequence diagram shows the process of a "Student" submitting live occupancy and noise feedback. It authenticates to be secure; if the user is authenticated, the AuthController verifies their credentials and creates a session. Once authenticated, the FeedbackController receives the submitted data, executes the insertFeedback() command in the System Database, and returns a success confirmation to the student.

3. View Live Occupancy Sequence Diagram



This sequence diagram shows the read only process of checking current library busyness. A "Student" requests the live occupancy status via the LibTrackerUI. The request is going to the OccupancyService, which directly fetches the current dataset from the System Database. The formatted occupancy data is then seamlessly displayed to the user. This flow is kept lightweight for quick access without requiring authentication.

4. Reserve Study Area Sequence Diagram



This sequence diagram shows the complex workflow for reserving a study area. Similar to the feedback process, it uses a block to be sure the "Student" is logged in via the AuthController. After authentication, the ReservationController makes a database interaction: it first calls checkAvailability() to ensure the seat is free, and after receiving an "available" response, it continues to saveReservation(). Finally, a reservation confirmation is sent back to the user's interface.

1.5 Selected Design Pattern

Selected Pattern: Singleton Design Pattern

Why it was chosen and Context:

In the LibTracker architecture, multiple controllers require constant and simultaneous interaction with the SystemDatabase. If the application were to create a brand new database connection object for every single user request, it would waste memory resources and inevitably cause data synchronization problems. The Singleton design pattern was specifically chosen to solve this problem by ensuring that only one shared instance of the SystemDatabase class exists during the entire application lifecycle.

How it is applied, Application in Context

The pattern is implemented by overriding the built in object creation method in the backend. For example, when a student submits an occupancy report, the FeedbackController requests a database object to save the data. Instead of opening a new connection, the SystemDatabase class checks its state. If an active connection instance already exists, it returns that exact instance; if not, it creates it. This approach effectively avoids memory leaks and keeps all database operations fully synchronized across different active users.

Code Snippet Illustration

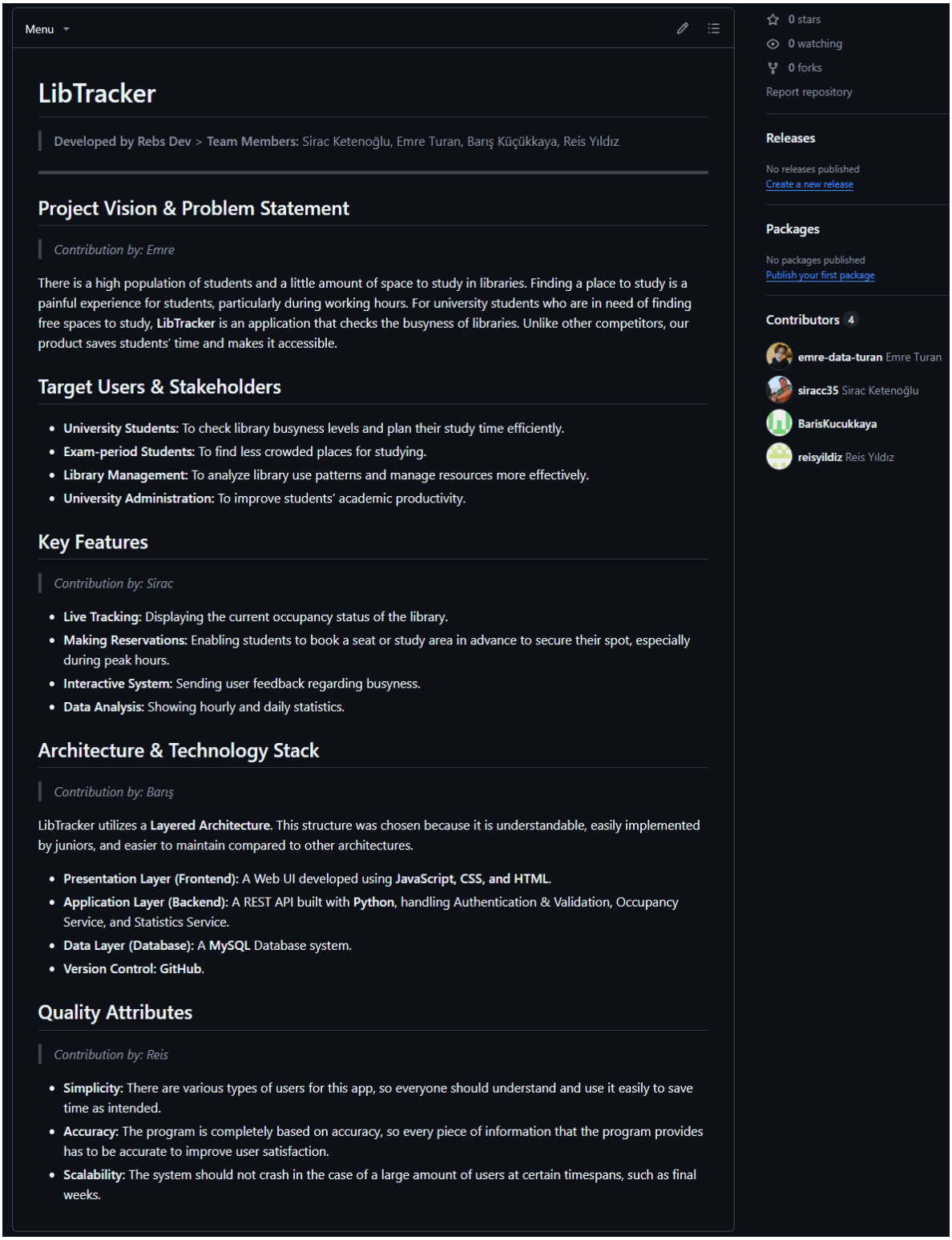
Below is the Python implementation demonstrating how the Singleton pattern controls the instantiation of the SystemDatabase:

```
class SystemDatabase:
    _instance = None

    def _new_(cls):
        if cls._instance is None:
            cls.instance = super().new_(cls)
        return cls._instance

    def insert_feedback(self, user_id, message):
        print("Data saved for " + user_id + ": " + message)

class FeedbackController:
    def submit(self, user_id, occupancy):
        db = SystemDatabase()
        db.insert_feedback(user_id, "Occupancy: " + str(occupancy))

controller1 = FeedbackController()
controller1.submit("Student_1", 85)
controller1.submit("Student_2", 20)

db1 = SystemDatabase()
db2 = SystemDatabase()

print("Are both database objects the same?")
print(db1 is db2)
```

1.6 GitHub Repository

Github Repository Link:

https://github.com/emre-data-turan/LibTracker.git



Menu

LibTracker

> Developed by Rebs Dev > **Team Members:** Sirac Ketenoğlu, Emre Turan, Barış Küçükkaya, Reis Yıldız

Project Vision & Problem Statement

> *Contribution by: Emre*

There is a high population of students and a little amount of space to study in libraries. Finding a place to study is a painful experience for students, particularly during working hours. For university students who are in need of finding free spaces to study, **LibTracker** is an application that checks the busyness of libraries. Unlike other competitors, our product saves students' time and makes it accessible.

Target Users & Stakeholders

- **University Students:** To check library busyness levels and plan their study time efficiently.
- **Exam-period Students:** To find less crowded places for studying.
- **Library Management:** To analyze library use patterns and manage resources more effectively.
- **University Administration:** To improve students' academic productivity.

Key Features

> *Contribution by: Sirac*

- **Live Tracking:** Displaying the current occupancy status of the library.
- **Making Reservations:** Enabling students to book a seat or study area in advance to secure their spot, especially during peak hours.
- **Interactive System:** Sending user feedback regarding busyness.
- **Data Analysis:** Showing hourly and daily statistics.

Architecture & Technology Stack

> *Contribution by: Barış*

LibTracker utilizes a **Layered Architecture**. This structure was chosen because it is understandable, easily implemented by juniors, and easier to maintain compared to other architectures.

- **Presentation Layer (Frontend):** A Web UI developed using **JavaScript, CSS, and HTML**.
- **Application Layer (Backend):** A REST API built with **Python**, handling Authentication & Validation, Occupancy Service, and Statistics Service.
- **Data Layer (Database):** A **MySQL** Database system.
- **Version Control: GitHub**.

Quality Attributes

> *Contribution by: Reis*

- **Simplicity:** There are various types of users for this app, so everyone should understand and use it easily to save time as intended.
- **Accuracy:** The program is completely based on accuracy, so every piece of information that the program provides has to be accurate to improve user satisfaction.
- **Scalability:** The system should not crash in the case of a large amount of users at certain timespans, such as final weeks.

0 stars
0 watching
0 forks
Report repository

Releases

No releases published
Create a new release

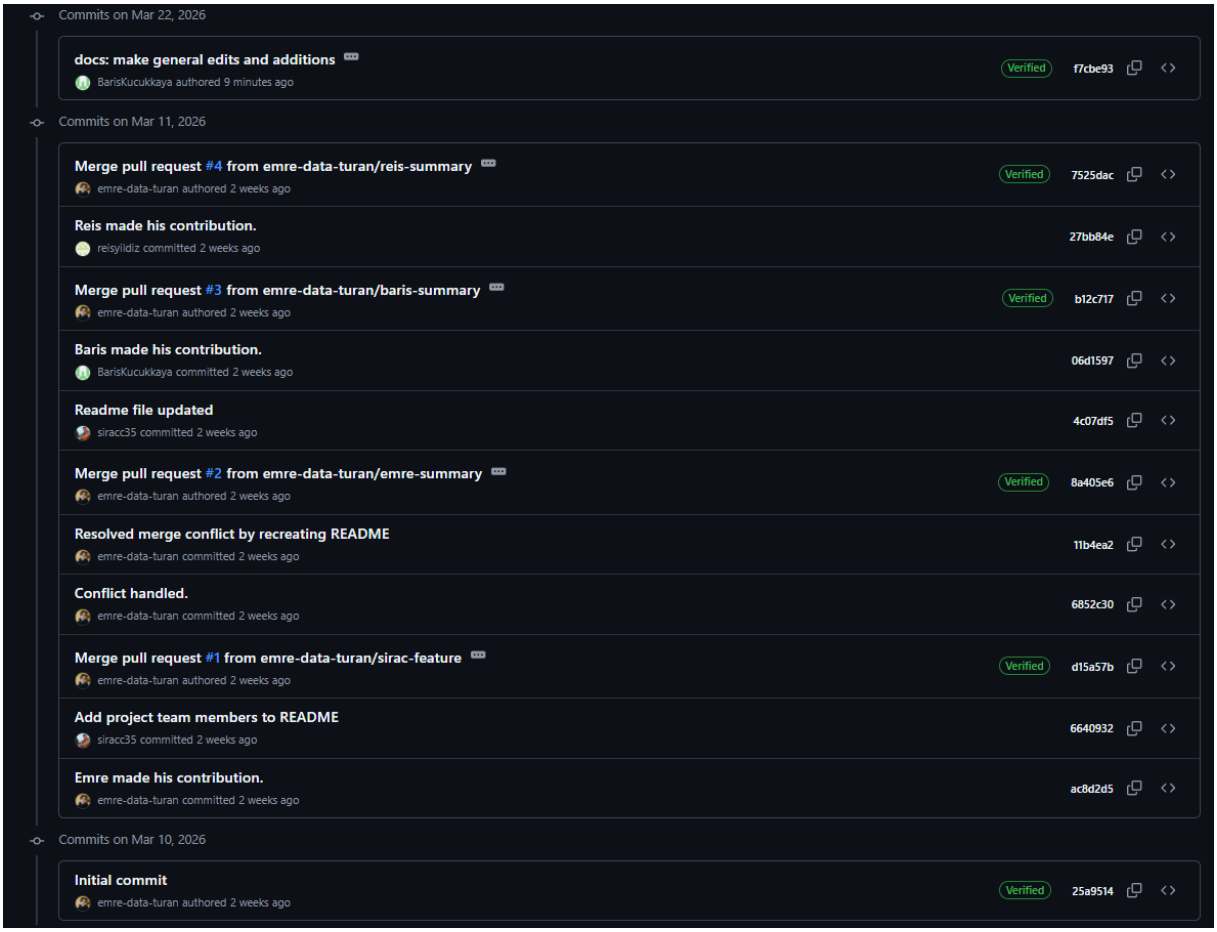
Packages

No packages published
Publish your first package

Contributors 4

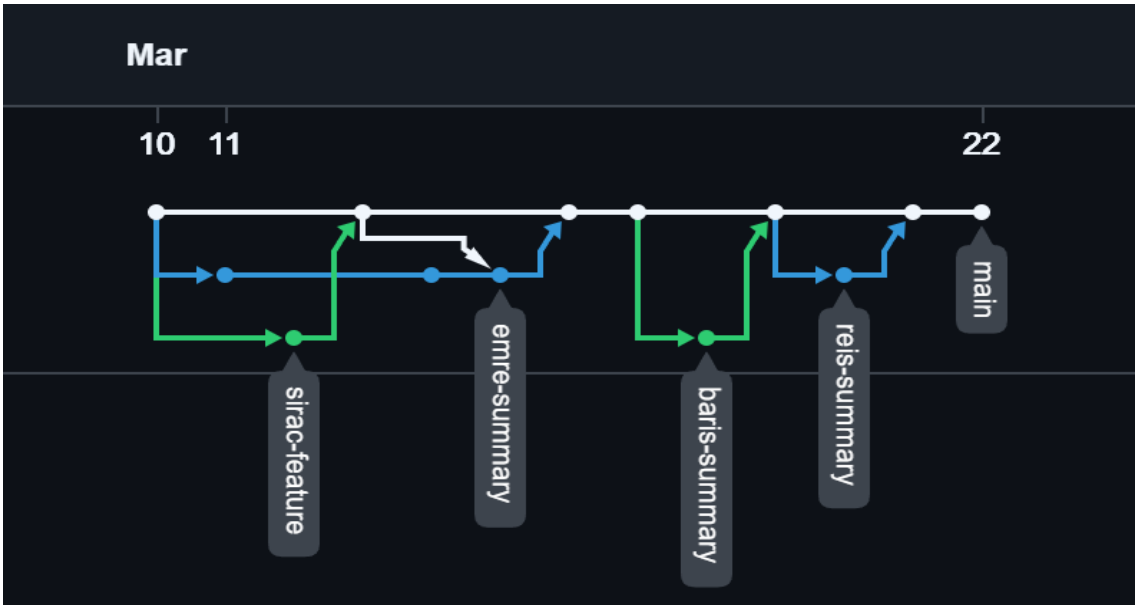
emre-data-turan Emre Turan
siracc35 Sirac Ketenoğlu
BarisKucukkaya
reisyildiz Reis Yıldız

Commit History:

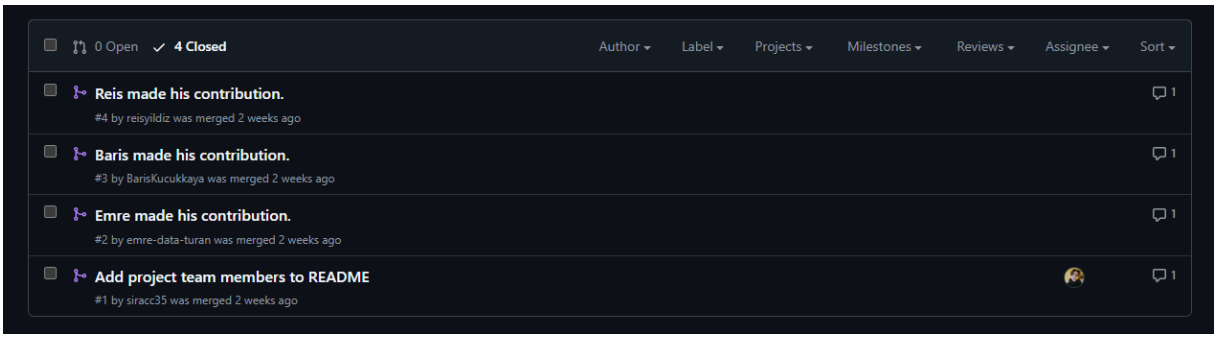



```
*   7525dac (origin/main, origin/HEAD) Merge pull request #4 from emre-data-
turan/reis-summary
|\
| * 27bb84e (origin/reis-summary) Reis made his contribution.
|/
*   b12c717 Merge pull request #3 from emre-data-turan/baris-summary
|\
| * 06d1597 (origin/baris-summary) Baris made his contribution.
|/
* 4c07df5 Readme file updated
*   8a405e6 Merge pull request #2 from emre-data-turan/emre-summary
|\
| *   11b4ea2 (HEAD -> emre-summary, origin/emre-summary) Resolved merge conflict
by recreating README
| |\
| |/
|/|
* |   d15a57b (main) Merge pull request #1 from emre-data-turan/sirac-feature
|\ \
| * | 6640932 (origin/sirac-feature) Add project team members to README
|/ /
| * 6852c30 Conflict handled.
| * ac8d2d5 Emre made his contribution.
|/
* 25a9514 Initial commit
```

Branching Strategy:



Pull Requests:



Folder Structure:

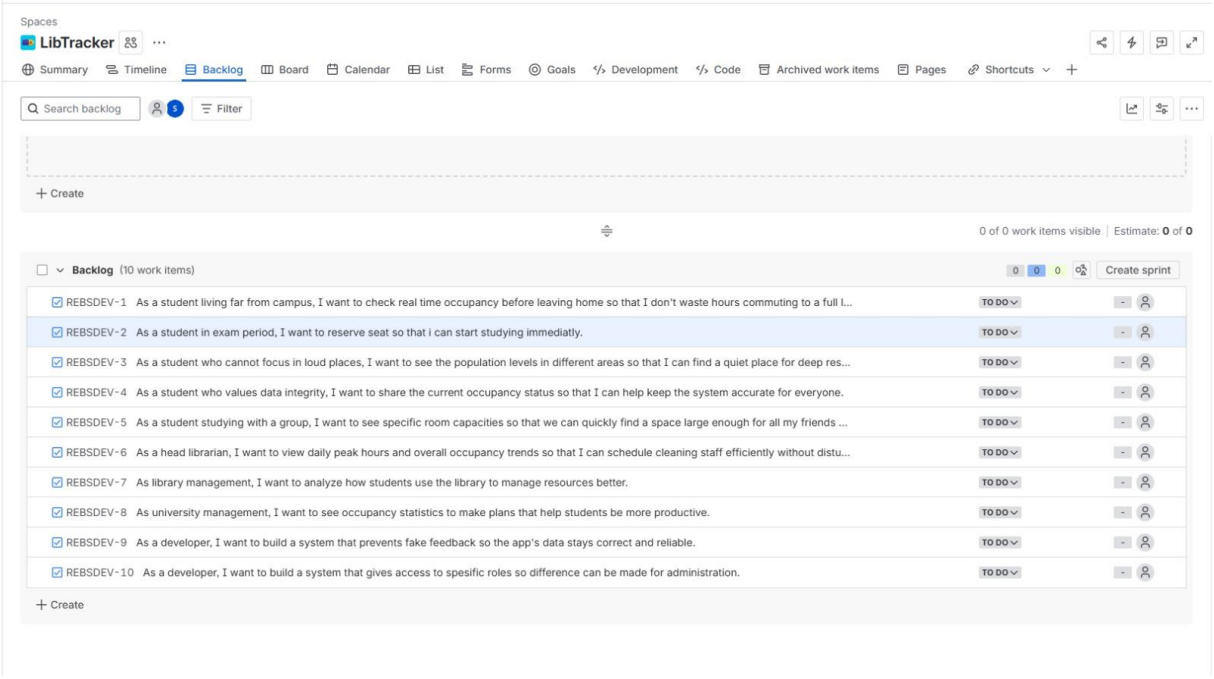
```
LibTracker/
├── frontend/              # Presentation Layer (HTML, CSS, JS)
├── backend/               # Application Layer (Python REST API)
│   ├── controllers/       # Bridges UI and logic (Auth, Feedback, Reservation, Stats)
│   ├── services/          # Core business logic (Occupancy, Statistics)
│   ├── models/            # System entities (User, Library, StudyArea,etc.)
│   └── config/            # Database configuration (Singleton Design Pattern)
├── database/              # Data Layer (MySQL setup scripts)
└── README.md
```

*(represents the **planned** folder structure)*

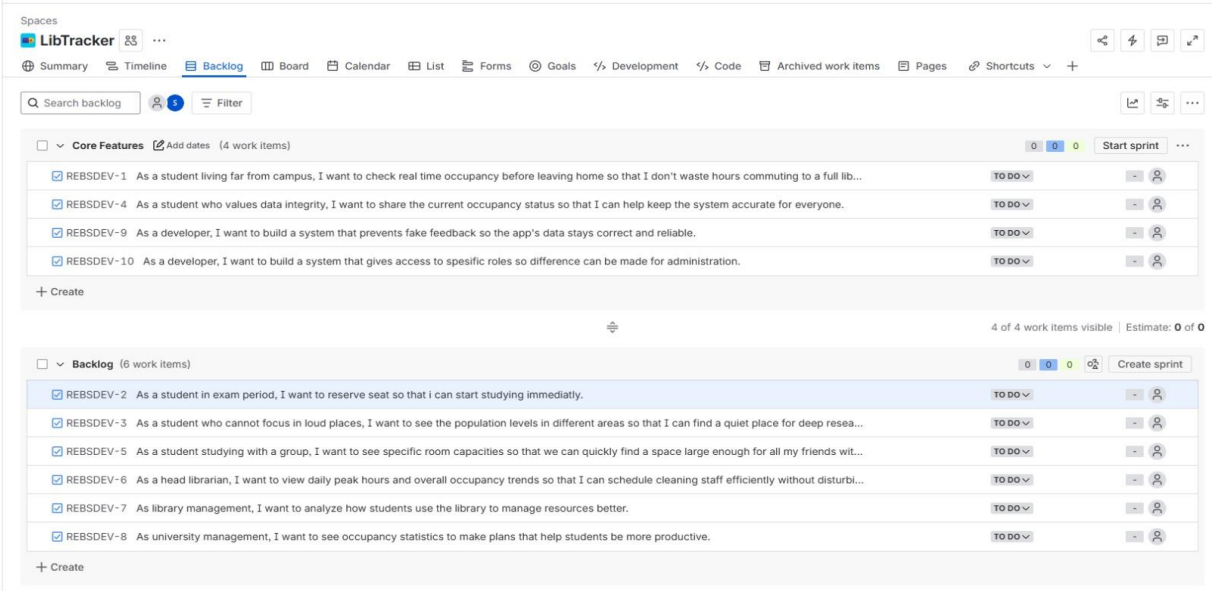
1.7 Jira Project Management

Tool Used

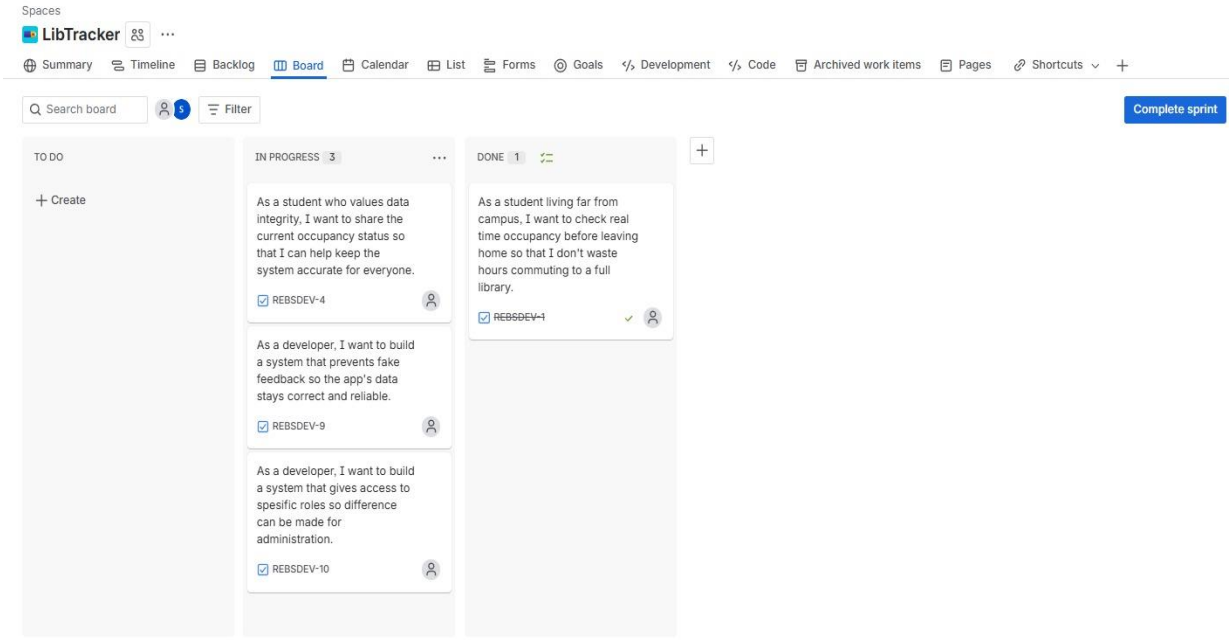
Jira was used as the project management tool to organise tasks, track progress, and manage development.



The backlog includes all user stories derived from personas and scenerios and prioritised based on importance.



Sprint 1 focuses on implementing core system features such as authentication, occupancy tracking, and feedback validation.



Tasks are actively tracked using statuses such as To Do, In Progress, and Done to monitor development progress.